

OSSP 6

Question:

1. Create a client-server application using Named Pipes (FIFOs). The client sends messages and the server processes and responds to them. Analyze FIFO behavior when multiple clients communicate simultaneously.

Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSR1. Demonstrate asynchronous event handling and explain the role of signal handlers.

Soution:

```
ubuntu@ubuntu:~$ nano server_fifo.c
ubuntu@ubuntu:~$ nano client_fifo.c
ubuntu@ubuntu:~$ gcc server_fifo.c -o server
ubuntu@ubuntu:~$ gcc client_fifo.c -o client
ubuntu@ubuntu:~$ ./server

I

^C
ubuntu@ubuntu:~$ ./client

^C
ubuntu@ubuntu:~$ nano signals.c
ubuntu@ubuntu:~$ gcc signals.c -o signals
ubuntu@ubuntu:~$ ./signals
Process ID: 8479
Waiting for signals...
```

Server_fifo.c:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <unistd.h>
```

```
#include <fcntl.h>
```

```
#include <sys/stat.h>
```

```
#include <string.h>
```

```
#define FIFO1 "client_to_server"
```

```
#define FIFO2 "server_to_client"
```

```
int main(){  
    char msg[100];  
  
    mkfifo(FIFO1, 0666);  
    mkfifo(FIFO2, 0666);  
  
    int fd_read = open(FIFO1, O_RDONLY);  
    read(fd_read, msg, sizeof(msg));  
  
    printf("Server received: %s\n", msg);  
  
    close(fd_read);  
  
    char reply[] = "Message processed by server";  
  
    int fd_write = open(FIFO2, O_WRONLY);  
    write(fd_write, reply, strlen(reply)+1);  
  
    close(fd_write);  
  
    return 0;  
}
```

client_fifo.c:

```
#include <stdio.h>
```

```
#include <stdlib.h>

#include <unistd.h>

#include <fcntl.h>

#include <string.h>


#define FIFO1 "client_to_server"

#define FIFO2 "server_to_client"


int main(){

    char msg[] = "Hello Server";

    char reply[100];


    int fd_write = open(FIFO1, O_WRONLY);

    write(fd_write, msg, strlen(msg)+1);

    close(fd_write);


    int fd_read = open(FIFO2, O_RDONLY);

    read(fd_read, reply, sizeof(reply));


    printf("Server Response: %s\n", reply);


    close(fd_read);


    return 0;

}
```

Signals.c:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <signal.h>
```

```
#include <unistd.h>
```

```
void handler(int sig)
```

```
{
```

```
    if(sig == SIGINT)
```

```
        printf("\nSIGINT received (Ctrl+C)\n");
```

```
    if(sig == SIGTERM)
```

```
        printf("\nSIGTERM received\n");
```

```
    if(sig == SIGUSR1)
```

```
        printf("\nSIGUSR1 received\n");
```

```
}
```

```
int main()
```

```
{
```

```
    signal(SIGINT, handler);
```

```
    signal(SIGTERM, handler);
```

```
    signal(SIGUSR1, handler);
```

```
    printf("Process ID: %d\n", getpid());
```

```
    printf("Waiting for signals...\n");
```

```
while(1)
    pause();

return 0;
}
```